UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 04

Representaciones Binarias

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Secuencia de clase **2**
2.1 Apertura — Corrección de ejercicios (10 min) 2
2.2 Complemento a 2 (15 min) 2
2.3 ASCII (10 min) 3
2.4 IEEE 754 (10 min) 3

3 Preguntas frecuentes **4**

4 Material de la sesión **5**

5 Rúbrica **5**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Calcular el rango de valores representables con n bits, con y sin signo.
2. Obtener la representación en complemento a 2 de un entero negativo y verificarla.
3. Interpretar una secuencia de bytes como texto ASCII y viceversa.
4. Distinguir mayúsculas de minúsculas a nivel de bit en ASCII.
5. Describir la estructura de un número flotante IEEE 754 y explicar por qué ocurren errores de redondeo.

50 minutos. Sugerido: 10 min corrección ejercicios sesión 03 + 15 min complemento a 2 + 10 min ASCII + 10 min IEEE 754 + 5 min cierre.

Secuencia de clase

- **Apertura — Corrección de ejercicios (10 min)**

Dedica los primeros minutos a cerrar la sesión anterior. Los ejercicios 5 y 6 son los que más trabajo dan; revísalos en el pizarrón mostrando el camino vía binario.

Los dígitos disponibles son $\{0, 1, 3, 4, 5, 6, 8, 9\}$ (8 símbolos = base 8). Renombrando $0 \to 0, 1 \to 1, 3 \to 2, 4 \to 3, 5 \to 4, 6 \to 5, 8 \to 6, 9 \to 7$, Carina cuenta en base 8. El milésimo número:

$$1000_{10} = 1750_8 \xrightarrow{\text{remap}} \mathbf{1960}$$

Verificación: $1 \times 512 + 7 \times 64 + 5 \times 8 + 0 = 512 + 448 + 40 = 1000$. ✓

Si algún alumno llegó a base 8 por sí solo, aprovecha para reconocerlo: es la parte más creativa del problema.

- **Complemento a 2 (15 min)**

Antes de explicar el método, lanza la pregunta:

"Si tengo 8 bits y quiero guardar números negativos, ¿cómo lo haría? ¿Cuál bit usaría para el signo?"

La respuesta intuitiva del grupo suele ser "el bit más a la izquierda como signo, el resto como magnitud" (signo-magnitud). Acepta esa respuesta y luego muestra el problema: con signo-magnitud existirían **dos ceros** (`00000000` y `10000000`), lo que complica la aritmética del procesador. El complemento a 2 elimina ese problema y simplifica los circuitos de suma.

Desarrolla los tres puntos en pizarrón:

1. La tabla de patrones de 8 bits con su valor. Énfasis en `10000000`=-128, `11111111`=-1.
2. El algoritmo invertir+1 con un ejemplo (-6, -50).
3. El rango general -2^{n-1} a $+2^{n-1}-1$ y por qué es asimétrico.

Pregunta: *"Si niego -128, ¿qué obtengo?"* Invertir `10000000` = `01111111`, sumar 1 = `10000000`. Se regresa a -128. Este es el único número que no tiene par positivo. Los alumnos suelen encontrar esto perturbador — aprovéchalos: es un bug real en software que asume simetría de rango.

- ## ASCII (10 min)

Conecta directo con lo anterior:

"Ya vimos cómo los números enteros viven en la memoria. ¿Cómo vive el texto?" La respuesta es sencilla: también como números. Un carácter = un byte = un entero de 8 bits. ASCII es el acuerdo de qué número corresponde a qué carácter.

Muestra la tabla A–Z en el pizarrón o proyector. Puntos clave:

1. Rango mayúsculas: 65 (A) a 90 (Z), es decir `0x41`–`0x5A`.
2. Rango minúsculas: 97 (a) a 122 (z), es decir `0x61`–`0x7A`.
3. Diferencia exacta de 32 = bit 5 activado.
4. Ejemplo en vivo: escribe tu nombre en hex. El ejemplo de clase fue "MAR" $\rightarrow$ `4D 41 52`.

Pide a un voluntario que diga sus iniciales. Calcula con el grupo el hex de cada letra. Es inmediato con la tabla y hace el concepto muy concreto. También funciona el ejercicio inverso: proyecta una secuencia de bytes y pregunta qué dice.

- ## IEEE 754 (10 min)

Los alumnos declararán `float` y `double` desde la sesión siguiente. Si no saben que los flotantes tienen errores de redondeo inherentes, los bugs que aparecerán en comparaciones y en acumulación de errores serán completamente opacos para ellos. No es necesario que comprendan la codificación completa; basta con que entiendan **por qué** `0.1 + 0.2 != 0.3`.

Secuencia sugerida:

1. **Factores primos de la base** (arranque conceptual). Pregunta: *"¿Por qué $\frac{1}{3}$ es periódica en decimal pero $\frac{1}{2}$ no?"* Guía al grupo a la respuesta: 2 divide a 10, pero 3 no. Generaliza: $\frac{1}{n}$ es exacta en base b cuando n solo tiene factores primos que también divide a b. En base 2, el único factor es 2, así que solo $\frac{1}{2^k}$ es exacta. Esto **no es culpa de la computadora**; es matemática pura.

2. **Algoritmo de multiplicación por 2** para convertir fracciones. Desarrolla en pizarrón $\frac{1}{4}$ (exacto) y $\frac{1}{3}$ (periódico). Con $\frac{1}{3}$ el grupo verá el ciclo aparecer naturalmente: en el segundo paso el estado de la parte fraccionaria vuelve al inicio, y se ve por qué el patrón 01 se repite indefinidamente.

3. 0.1_{10} **en binario**: aplica el mismo algoritmo. Muestra que el ciclo comienza en el segundo paso y produce el patrón $\overline{0011}$, de donde $0.1_{10} = 0.0\overline{0011}_2$.

4. **Estructura IEEE 754**: signo (1 bit), exponente (8 bits), mantisa (23 bits) para `float`. La mantisa trunca la secuencia infinita, introduciendo el error.

5. **Regla práctica**: nunca comparar flotantes con ==. Usa `|a-b| < epsilon`.

Después de mostrar que $\frac{1}{3} = 0.\overline{01}_2$, pregunta:
"¿Cuántos bits necesitarías para representar exactamente $\frac{1}{3}$?" Respuesta: infinitos. Esto ancla por qué el error de redondeo es **inevitable** y no un bug que alguien pueda corregir añadiendo más bits.

Si tienes el cuaderno Jupyter abierto, ejecuta:

```
#include <iostream>
int main(){
    double a = 0.1 + 0.2;
    std::cout << (a == 0.3) << "\n";    // imprime 0
    std::cout << a << "\n";             // 0.3
    std::cout.precision(17);
    std::cout << a << "\n";             // 0.30000000000000004
}
```

El resultado de precisión 17 produce el efecto "wow" que fija el concepto mejor que cualquier explicación.

Preguntas frecuentes

"¿Por qué el rango de `char` en C++ va de -128 a 127 y no de -127 a 127?"
Porque `char` es un entero de 8 bits en complemento a 2. El patrón `10000000` se asigna a -128 (no existe -0). Nota adicional: en C++ `char` puede ser `signed` o `unsigned` según la implementación; usa `int8_t` / `uint8_t` si necesitas garantías.

"¿Qué pasa con los acentos y la ñ en ASCII?"
No existen en ASCII estándar (7 bits). El ASCII "extendido" de 8 bits tiene varias codificaciones incompatibles (ISO-8859-1, Windows-1252, etc.). La solución moderna es **UTF-8**, que representa los primeros 128 caracteres igual que ASCII y usa 2–4 bytes para los demás.

"¿Cuándo uso `float` y cuándo `double`?"
Por defecto usa `double`: tiene más precisión (52 bits de mantisa vs. 23) y en CPUs de 64 bits no es más lento. `float` se usa en gráficos, audio y aprendizaje automático donde el volumen de datos hace que el ahorro de memoria importe.

"¿Existe un estándar para enteros negativos alternativo al complemento a 2?"
Históricamente sí: signo-magnitud y complemento a 1. Desde C++20 el estándar exige oficialmente complemento a 2 para todos los enteros con signo. En la práctica, todas las CPUs modernas ya lo usaban.

Material de la sesión

- **Notas manuscritas originales**: `materialInicial/sesion04/`
- **Notas alumno**: `notas_alumno/pdf/sesion04_representaciones_binarias.pdf`

Rúbrica

Criterio	Puntos	Evidencia
Convertir entero negativo a complemento a 2 (proceso)	3	pasos visibles
Convertir complemento a 2 a decimal (leer negativo)	2	resultado correcto
Decodificar 3 caracteres ASCII desde hex	2	tabla usada correctamente
Codificar nombre/iniciales en hex ASCII	2	coincide con tabla
Identificar campo signo/exponente/mantisa en `float`	1	estructura correcta
Total	10	